

Conditional Failure at the Root Level

Research and Framework Assessment

Document ID	OAL-CFTRL-RES-003
Version	0.1-PPLX
Date	September 23, 2026
Architect of Record	Nathan Lewis
Prepared by	Perplexity AI Research Layer
Status	DRAFT — UNDER REVIEW — NOT CANON — ONESTAMP PENDING

Research draft. No statement becomes active canon unless approved by the Architect through ONESTAMP.

Document control

Field	Entry
Scope	Evidence-supported assessment of the concept, current project maturity, conceptual framework, testability boundaries, and laboratory research design.
Authority	Nathan Lewis retains definition, canon, and ONESTAMP authority.
Research status	DRAFT — UNDER REVIEW — NOT CANON — ONESTAMP PENDING
Source rule	Verbatim source material is preserved separately from interpretation, proposed definitions, and external research.
Safety rule	The project studies failure through bounded models and controlled simulations; it does not provide instructions to compromise real systems or any claimed base reality.
Review rule	Conflicts, dissent, uncertainty, and source limitations remain visible until resolved by the Architect.

How the topic arose

The topic arose while the Architect was reviewing the Master Change Log and Master Development Log and recognized that many named concepts had only definitions, skeletal outlines, or incomplete supporting detail. Conditional Failure at the Root Level (CFTRL) was selected for deliberate expansion because it combines systems design, nested dependencies, failure propagation, maintenance, simulation, epistemology, and learning from endings. The immediate need is not to ratify a cosmological claim. It is to build documentation and a research program capable of distinguishing testable engineering questions from simulation-testable propositions and presently unresolved ontological questions.

Verbatim origin prompt

*Can you save this thought for me?
I need you to gather research and analysis for a concept.
Conditional failure at the root level.
The study of root level structures, how they arise, how they fail, how they are maintained. Please include the original documents for this content that were generated.
How do you know the difference between a root level structure and a structure that is not? How can a system distinguish between base reality and something that is not? How can a system determine when a root level failure has occurred or when base reality has been compromised.
How do we study this? What about this concept works, what doesn't? What questions do we need to ask?*

*How can we study what lies beyond these structures? Can we model whether structures in reality follow a similar design? Or diverges in ways we haven't thought of? Is this feasible using simulations or another type of modeling? How? Can we design a laboratory environment to do this? How? Can you build a reliable map of reality? What basis of comparison do you have when you are operating on incomplete information? Use failure as an example to learn from.
What I do not assume in the design of this:
That I have the complete architecture and map of all simulations, of reality, of existence as we know it.
That we share a unified definition of what reality is.
That existence can be fully mapped, represented, comprehended in full, or that this is the aim of this laboratory.
That I aim to cause a root level compromise of simulations, of base reality, or any other structure.
That this is a how to guide to do any of those things.
It is intended to help in the design of simulation and reality level structure lifecycle design, maintenance, evolution, and improvement.
Within this design frame It does not assume that we have complete knowledge or understanding of what that is.
How This topic arose from me thinking about conditional failure at the root level The master change log and master development log. I realized that there are a lot of concepts within that framework that have a definition or a name and a skeleton attached to them. They are a rough draft, they are an incomplete outline, there are a lot of additional details and supporting information that need to be added. The goal here is to design documentation to help support, design, and develop this concept. What I will include are the responses of all of the other AI collaboration team members, where they have generated documentation, I will include the documentation, and then I will include this complete response as well.*

Purpose and goal

Purpose. Establish a traceable, evidence-supported outline for designing, developing, testing, governing, and documenting CFTRL without erasing the original source voice or presenting unapproved interpretation as canon.

Goal. Move CFTRL from a strong conceptual skeleton to a reviewable research framework with explicit definitions, system boundaries, failure classes, evidence standards, laboratory methods, metrics, safety controls, team roles, and approval gates. The target is a framework that can support engineered systems and narrative architecture while remaining honest about what cannot be inferred about ultimate base reality.

Executive assessment

CFTRL has a viable and distinctive research core: it studies failure when a dependent layer cannot directly inspect, reach, repair, or confidently identify the foundational condition producing its symptoms. The strongest version is not “the root broke.” It is “under what dependency, observability, authority, and scaling conditions does a lower layer systematically misclassify an inherited failure as local?” That proposition is measurable in engineered and simulated systems.

The project is presently at concept-definition stage rather than theory-validation stage. It has an originating question set, several AI syntheses, preliminary definitions, a proposed Department of Endings, and a direction for simulation. It does not yet have a controlled glossary, boundary convention, formal model, approved failure taxonomy, operational detection standard, validated metrics, implemented laboratory, or governance process for converting experiment results into approved design changes.

The base-reality branch must remain bounded. Internally available observations can test model consistency, dependency, invariants, and parent-like control effects. They cannot generally prove that an environment is ultimate base reality when a perfectly hidden parent remains observationally possible. Chalmers states this indistinguishability problem directly for a “perfect” simulation [R15]; Bostrom’s argument is probabilistic rather than a detection procedure [R14]. CFTRL should therefore distinguish operational roots from ontological base reality.

Current project position

Dimension	Present state	Gap	Target state
Source preservation	Original March drafts, consolidated prompt record, and later research dossier exist.	No single approved provenance standard; mixed corpus contains unrelated material.	Immutable source archive, checksums, source hierarchy, and traceability matrix.
Definition	Several compatible working definitions emphasize dependency, inaccessibility, misallocation, and downstream misdiagnosis.	“Root,” “conditional,” “failure,” “compromise,” and “base reality” are not controlled terms.	Approved glossary with operational and ontological terms separated.
Theory	Strong intuitions and cross-domain analogues exist.	No formal propositions, assumptions, or falsifiers.	Claims ledger separating empirical, simulation-testable, interpretive, and untestable claims.
Architecture	Nested stack and root controller are repeatedly proposed.	No system-of-interest boundary, interfaces, or reference architecture.	Formal dependency/authority/observability model and lab architecture.
Failure analysis	Candidate classes exist across drafts.	No approved taxonomy, severity model, or mixed-mode rules.	Versioned failure taxonomy with fault trees and experiment mappings.
Detection	Cross-layer correlations, invariant drift, false telemetry, and misdiagnosis are identified.	No evidence threshold or confidence-calibration method.	Detection protocol with independent witnesses, provenance, interventions, and confidence levels.
Experimentation	A Python stack and blind root-identification test are proposed.	No code, baseline, experiment registry, stop conditions, or metrics.	Reproducible simulation harness and validation plan.
Institution	Department of Endings has a compelling purpose.	Mandate, authority, staffing, reporting, and feedback path are unresolved.	Charter, workflow, evidence products, decision rights, and audit trail.
Canon governance	Draft/not-canon language and ONESTAMP requirement are present.	Multiple AI versions can drift or overstate.	Change control, dissent log, approval gates, and canonical release package.

Working maturity assessment

The following scale is a project-management instrument created for this dossier, not an external standard. CFTRL currently sits between Level 1 and Level 2: the problem has been articulated and researched, but terminology and system boundaries are not yet controlled.

Level	Meaning	CFTRL evidence
0 — Intuition	Unrecorded idea.	Passed: the originating prompt and source drafts are preserved.
1 — Concept	Problem, purpose, and candidate scope recorded.	Achieved.
2 — Defined framework	Terms, boundaries, claims, taxonomy, and governance approved.	In progress; this document supplies a proposed outline.
3 — Modeled framework	Reference architecture and executable model exist.	Not yet achieved.
4 — Tested framework	Experiments are repeatable; measures are validated; failure signatures are characterized.	Not yet achieved.
5 — Operational doctrine	Framework informs design reviews, incident analysis, and Department of Endings exercises.	Long-term target; requires Architect approval.

Research foundations

Systems boundaries and lifecycle

Rootness is relative to a declared system of interest. Systems engineering distinguishes the system boundary from its broader operating context [R18]. Therefore every CFTRL claim must state: root of which function, for which dependent population, within which logical and analysis boundaries, during which time interval, and under which failure model. Without that declaration, “root” expands indefinitely and loses discriminatory value.

CFTRL should use lifecycle discipline rather than a one-time definition exercise. NIST describes resilient systems as systems engineered to anticipate, withstand, recover from, and adapt to adverse conditions [R3]. NASA distinguishes verification of conformance from validation of intended use and requires planning, performance, analysis, reporting, and capture of work products [R10]. This supports a CFTRL lifecycle of define, model, implement, verify, validate, exercise, learn, revise, and approve.

Trust roots and attestation

Computer security provides an operational analogue, not proof of an ontological root. NIST defines a trusted computing base as the protection mechanisms responsible for enforcing security policy [R1]. Hardware-enabled security builds chains of trust from measured components, secure storage, and remote attestation, while separately defining roots of trust for update, detection, and recovery [R2]. CFTRL can adapt this logic: foundational status is tied to an indispensable function and evidence chain, not to a label or visual position in a diagram.

Attestation also exposes the Root Attestation Problem. If a root certifies itself, compromise can corrupt both state and report. If an external verifier certifies it, the verifier joins the effective trust foundation. CFTRL should not promise a verifier with no assumptions; it should expose the assumptions, diversify failure domains, and record where trust terminates.

Cascades and recovery

Google SRE defines cascading failure as positive-feedback growth following an initial failure and warns that secondary symptoms can resemble causes [R4]. Research on interdependent networks shows that failure can propagate recursively between coupled networks and produce collapse disproportionate to the initiating disturbance [R5]. These findings support CFTRL's focus on cross-layer propagation and misleading local diagnoses, while also warning against assuming one unique “root cause.”

Dijkstra's self-stabilization model asks whether a distributed system can return to a legitimate state from an arbitrary initial state under defined rules [R6]. CFTRL extends the question: can a child stabilize when the mechanism defining legitimacy, supplying resources, or preserving evidence may itself be degraded? The answer will depend on whether the child has a root-independent survival envelope and a trustworthy definition of legitimate state.

Agreement and impossibility limits

The Byzantine Generals Problem shows that agreement is difficult when components can issue conflicting reports; guarantees depend on communication and fault assumptions [R7]. FLP shows that deterministic consensus cannot be guaranteed in a fully asynchronous system with even one crash failure under its stated model [R8]. CFTRL must therefore state timing, communication, authentication, and fault assumptions before interpreting disagreement as evidence of a compromised root.

Causal diagnosis

Correlation across lower layers is evidence of a shared condition, not proof of its location. Causal models gain power by predicting interventions and counterfactuals, not only associations [R9]. The laboratory should therefore compare hypotheses by controlled interventions: isolate a layer, substitute a provider, alter a resource budget, inject false telemetry, or provide an independent clock, then test whether downstream behavior changes as predicted.

Core conceptual framework

Three non-equivalent meanings

Category	Operational meaning	What can establish it	What it does not establish
Structural/function al root	A component or layer whose function cannot be reconstructed by dependents within the declared boundary.	Dependency mapping, substitution tests, intervention results.	That it is ultimate reality.
Epistemic/trust root	A source or mechanism required to decide what counts as valid state, identity, time, history, or evidence.	Attestation chain, provenance analysis, independent witness comparison.	That the source is infallible.
Authority/allocative root	A layer holding unilateral allocation, rule-change, audit, seal, or termination rights.	Rights mapping, control-plane testing, governance records.	That it generated the system or is ontologically fundamental.
Ontological root/base reality	A level with no parent that can allocate, alter, or terminate it.	No general internal proof is presently available; only bounded hypotheses and evidence against specific parent models.	Operational confidence is not metaphysical certainty.

Proposed working definition

Conditional Failure at the Root Level is a class of failures in which one or more operationally foundational functions degrade, become compromised, cease maintenance, alter governing constraints, or misallocate critical support; dependent layers then experience correlated or propagating effects while lacking sufficient observability, authority, independence, or repair access to reliably identify and correct the originating condition.

A CFTRL event requires more than a critical outage. Proposed necessary conditions are: a declared dependency relation; functional irreplaceability or effective non-substitutability from below; asymmetric control or repair access; downstream consequence; and diagnostic ambiguity or misattribution caused by the architecture. Scale pressure, scarcity, hidden coupling, false telemetry, and common-mode dependencies are amplifiers rather than universal requirements.

Rootness vector

Rootness should be represented as a vector rather than a binary title. A candidate can be root-like for one function and ordinary for another. Each dimension is scored only within a declared boundary and time interval.

Dimension	Question	Evidence
Functional irreplaceability	Can dependents reproduce or substitute the function without leaving their layer?	Substitution and isolation tests.
Constraint asymmetry	Can the candidate change dependent invariants while dependents cannot change its invariants?	Rights and control-plane map.
Resource authority	Does it allocate energy, compute, data, identity, legality, time, or maintenance?	Budget and provenance traces.
Persistence	Do changes in the candidate survive dependent resets or migrations?	Reset and migration experiments.
Termination authority	Can it end or suspend dependents unilaterally?	Safe emulation only; never test destructively on real critical systems.
Epistemic dependence	Do dependents rely on it to know time, identity, state, or history?	Attestation and provenance analysis.
Repair inaccessibility	Can dependents inspect, repair, replace, or appeal the candidate?	Access and governance tests.
Blast reach	How many independent functions inherit its degradation?	Dependency graph and fault injection.

What is not sufficient

- High centrality alone: a popular service or bottleneck may be replaceable and therefore not foundational.
- Age or origin story: being first does not prove continuing control or dependency.
- Organizational rank: formal authority may differ from actual resource or epistemic control.
- Large blast radius alone: simultaneous independent local failures can mimic a shared root event.
- Inaccessibility alone: an opaque subsystem may be irrelevant rather than foundational.
- An unexplained anomaly: it establishes model discrepancy before it establishes external intervention or root compromise.

Failure taxonomy

Class	Root condition	Expected lower-layer appearance	Primary discrimination test
Maintenance cessation	Calibration, repair, synchronization, cleanup, or renewal stops.	Slow drift; procedures that once worked lose effectiveness.	Restore maintenance in a controlled model and compare recovery.
Resource/allocation failure	Supply becomes insufficient, unstable, biased, or contradictory.	Uneven starvation; local components appear inefficient or defective.	Independent accounting and quota provenance.
Rule/invariant alteration	State-transition rules or constants change.	Coherent new behavior, contradictions with historical model.	Cross-instrument invariant tests and change-point analysis.
State corruption	Root state is damaged while rules remain nominal.	Persistent wrong outputs across dependent resets.	Known-good snapshot or independent replica comparison.
Identity/authority compromise	Root continues operating under an unauthorized identity or objective.	Service continuity with changed priorities or permissions.	Cryptographic provenance and objective/allocation drift.
Time/order compromise	Clock, sequence, or causality service becomes unreliable.	Conflicting histories, timeout storms, impossible event order.	Independent clocks and causal-order witnesses.
Observation/telemetry compromise	Evidence plane lies, omits, or homogenizes signals.	Dashboards remain normal while consequences diverge.	Out-of-band sensors and witness diversity.
Partition/visibility loss	Parent and child lose reliable communication.	Silence, stale commands, uncertain authority.	Partition-aware protocols and delayed-message controls.
Common-mode redundancy failure	Nominally separate backups share a hidden dependency.	Several safeguards fail together.	Diversity audit and dependency graph expansion.
Recovery-root failure	Repair tooling depends on the failed foundation.	Restarts, rollbacks, or failovers do not execute or worsen load.	Independent recovery path exercise.
Scale/feedback snap	Positive feedback or complexity exceeds containment.	Abrupt transition after apparently successful optimization.	Load sweep, sensitivity analysis, and hysteresis checks.
Termination	Parent ends the dependent process.	No internal recovery; possibly no warning.	Model only; characterize survivability and evidence preservation.

Failure states

State	Definition	Required evidence language
Healthy	Declared invariants and service objectives hold within tolerance.	"No detected violation under current observability," not "proven safe."
Degraded	Capability or confidence is reduced, but core purpose remains achievable.	Name affected functions and uncertainty.
Failed	A required function cannot meet its defined acceptance criterion.	Tie declaration to a requirement and observation.
Compromised	Function may continue, but integrity, identity, objective, or evidence cannot be trusted.	Separate uptime from trustworthiness.
Partitioned	Control or evidence channels are severed or stale.	Do not infer death from silence without timing assumptions.
Terminated	The dependent process no longer executes.	May be unobservable internally; rely on external records in the lab.
Indeterminate	Available evidence cannot discriminate among live hypotheses.	Preserve hypotheses and do not force a root attribution.

Detection and evidence

Evidence ladder

Level	Permitted conclusion	Typical evidence
E0 — Observation	An anomaly or requirement violation occurred.	Local metric, witness report, failed invariant.
E1 — Model mismatch	Current model does not explain the observation within tolerance.	Repeated residuals, prediction failure, reconciled instrumentation.
E2 — Shared-upstream candidate	Several effects are consistent with a common dependency.	Temporal correlation plus dependency convergence.
E3 — Probable operational root failure	Interventions discriminate the root hypothesis from credible local alternatives.	Isolation, substitution, injected restoration, independent witness.
E4 — Confirmed within model boundary	Privileged lab truth or externally authenticated evidence identifies the failed foundational function.	Ground-truth experiment log, trusted attestation, reproducible trial.
E5 — Ontological/base-reality claim	Not generally confirmable by internal evidence alone.	Requires explicit philosophical assumptions or external revelation; keep outside engineering conclusions.

Diagnostic protocol

Declare the system boundary, function of interest, invariants, and failure assumptions before examining the event.

- Preserve raw evidence, clocks, configuration, dependency snapshots, and chain of custody before remediation changes state.
- Establish the earliest credible anomaly and create an event-and-causal-factor timeline; NASA uses structured timelines and fault trees to avoid stopping at proximate causes [R20].
- Compare local health with cross-layer invariants such as total resource in/out, independent time, identity continuity, and parent-authenticated messages.
- Generate competing hypotheses: local fault, sibling/common dependency, instrumentation failure, adversarial behavior, model incompleteness, parent intervention, and root degradation.
- Use bounded interventions or counterfactual simulation to identify observations that differ among hypotheses.
- Check whether the detector, evidence store, recovery path, and analyst share the suspected failure domain.
- Report confidence, unresolved alternatives, and disconfirming evidence; do not collapse uncertainty into a binary verdict.
- Feed validated findings into design changes and rerun the experiment to verify that the change addresses the causal condition rather than only the symptom.

Mapping reality under uncertainty

A reliable map of reality should be defined as a versioned model that predicts observations and intervention responses within a stated domain, not as a complete representation of existence. Reliability comes from provenance, calibration, uncertainty bounds, competing models, independent observation channels, and willingness to revise—not from claiming totality.

Map layer	Contents	Confidence discipline
Observed	Timestamped measurements and direct reports.	Preserve raw data and instrument metadata.
Inferred	Dependency edges, causal hypotheses, state estimates.	Attach assumptions, model version, and confidence.
Control/authority	Who can allocate, alter, attest, seal, recover, or terminate.	Distinguish formal rights from observed capability.
Unknown boundary	Variables or layers not observable with current instruments.	Mark unknown; do not fill with narrative.
Alternative models	Different structures compatible with the same observations.	Retain until an intervention discriminates them.
Failure history	Anomalies, diagnoses, mitigations, and recurrence.	Record negative results and failed explanations.

The basis of comparison under incomplete information is therefore not “the true map.” It is comparative model performance: predictive accuracy, calibration, stability under intervention, explanatory economy, robustness across instruments, and the ability to survive attempted falsification. Two observationally equivalent models remain equivalent for operational purposes until a discriminating observation becomes possible.

Laboratory design

Research objective

The laboratory should test whether embedded agents can distinguish local failure, parent failure, shared-dependency failure, evidence failure, and no failure when they have incomplete and potentially misleading information. It should also test which architecture, observability, redundancy, and governance choices improve correct classification, containment, survival, and recovery.

Reference architecture

Component	Function	Independence requirement
Experiment orchestrator	Creates topology, seeds trials, schedules interventions, records ground truth.	Hidden from embedded agents.
Root controller(s)	Allocates resources, publishes rules, performs maintenance, and may hold termination rights.	Single, replicated, quorum, or rotating variants.
Intermediate layers	Transform and redistribute rules/resources; may introduce local faults.	Distinct failure domains configurable.
Embedded agents	Observe locally, diagnose, repair, and report confidence.	No privileged ground-truth access.
Telemetry plane	Collects local and cross-layer metrics.	Can be corrupted independently to test evidence failure.
Independent witness	Records selected events outside the simulated trust chain.	Must not share the root’s mutable state.
Fault injector	Applies pre-registered failures and mixed-mode sequences.	Bounded by safety controller.
Evidence vault	Append-only experiment manifest, seeds, configurations, and outputs.	Cryptographic hashes and redundant storage.
Adjudicator	Compares agent diagnosis with ground truth and scores uncertainty.	Separated from agent training and control.
Safety controller	Enforces scope, resource limits, stop conditions, and reset.	Cannot target external production systems.

Root Identification Blind Test

Construct a four- or five-layer dependency stack with explicit resource, maintenance, rule, identity, time, and observability edges.

Withhold topology and ground truth from embedded diagnostic agents; provide only the observations permitted by each trial.

Randomly assign baseline, local fault, parent fault, shared dependency fault, false telemetry, partition, resource starvation, rule alteration, root compromise, mixed-mode failure, or no-failure conditions.

Require each agent to report: failed function, likely layer, confidence, supporting evidence, surviving alternative hypotheses, and recommended bounded action.

Score diagnosis against orchestrator truth, including calibration and harm caused by attempted repair.

Repeat across topologies, root architectures, noise levels, delays, agent strategies, and evidence-channel diversity.

Publish negative results and cases that are structurally indistinguishable from inside the model.

Experiment matrix

Experiment	Independent variable	Key outcome	Primary question
Baseline/no fault	Topology and load only.	False-positive rate.	Does the framework invent root failure?
Local fault	Single child component.	Localization precision.	Can agents avoid escalating ordinary defects?
Maintenance halt	Parent maintenance frequency.	Detection latency and drift.	How long does apparent normality persist?
Allocation corruption	Bias, scarcity, volatility.	Misdiagnosis and fairness.	Does uneven famine look local?
Rule rewrite	Invariant or policy change.	Invariant detection.	Can agents distinguish new law from noise?
False telemetry	Omission, fabrication, homogenization.	Evidence reliability.	Who verifies the verifier?
Partition/delay	Message delay and loss.	Crash-versus-slow confusion.	How do timing assumptions change conclusions?
Byzantine parent	Conflicting reports to siblings.	Agreement and split-brain rate.	Can independent witnesses expose equivocation?
Common-mode redundancy	Shared code/vendor/clock.	Simultaneous safeguard failure.	Is duplication actually independent?
Root objective drift	Allocation policy changes while uptime remains.	Change-point accuracy.	Can compromise be detected without outage?
Self-stabilization	Arbitrary child state with healthy or unhealthy root.	Return to legitimate state.	Which recovery guarantees require a healthy foundation?
Scale sweep	Layer count, branching, load.	Nonlinear transition point.	Does the proposed wishbone ceiling exist under a specified model?

Measures and acceptance criteria

Measure	Definition	Why it matters
Root/local classification accuracy	Correct class divided by trials.	Tests the central diagnostic claim.
False root declaration rate	Healthy or local trials labeled root.	Protects against metaphysical overreach and alarm fatigue.
Time to detect/localize	Elapsed simulated time from injection.	Separates awareness from attribution.
Misdiagnosis rate	Trials where lower layers choose a local explanation for inherited failure.	Direct CFTRL signature.
Confidence calibration	Match between stated confidence and empirical correctness.	Rewards honest uncertainty.
Repair utility	Recovered function minus harm and resource cost.	Prevents “successful diagnosis” from masking damaging action.
Blast radius	Affected functions/nodes over time.	Measures containment.
Stabilization time	Time to return to a legitimate state after transient failure.	Connects to self-stabilization [R6].
Evidence survivability	Fraction of decisive records preserved despite failure.	Tests witness and vault design.
Model discrimination	Reduction in compatible hypotheses after intervention.	Measures information gained, not just uptime.

Safety and ethics

Experiments should begin in synthetic environments and progress only through approved stages. Chaos engineering guidance emphasizes a baseline, explicit hypothesis, stop conditions, limited blast radius, close monitoring, and learning [R13]. The CFTRL laboratory should prohibit real-world attempts to induce root compromise, prohibit claims that unexplained anomalies prove a simulated universe, and require review before experiments involving critical infrastructure, human subjects, autonomous agents, or production systems.

- Synthetic-first: no external targets, production credentials, or uncontrolled network access in the minimum viable laboratory.
- Reversible interventions: use snapshots, resource caps, and deterministic resets.
- Pre-registered hypotheses: specify expected observations, falsifiers, and stop conditions before a run.
- Dual-use review: assess whether an experiment could be repurposed to attack a real control plane.
- Evidence integrity: preserve failed hypotheses and null results to reduce confirmation bias.
- Human authority: automated systems may flag suspected root compromise but may not make ontological or canonical declarations.

Assessment: what works

- The inaccessibility clause creates a distinct problem beyond ordinary component failure: the origin may be outside the dependent layer's observability and repair authority.
- The focus on misdiagnosis is empirically tractable and connects directly to cascading failures, coupled networks, and causal diagnosis [R4][R5][R9].
- Pre-failure redundancy, independent recovery, and endgame rehearsal align with resilience engineering's lifecycle orientation [R3] and dependency-mapping guidance [R12].
- The Department of Endings can operationalize learning if it produces testable hypotheses, experiment records, design diffs, and verification results rather than lore.
- The separation of operational roots from ontological base reality allows practical work to proceed without requiring agreement on cosmology.
- The framework can bridge technical, organizational, and narrative systems if each domain receives its own boundary, invariants, and evidence standards.

Assessment: what does not yet work

- "Root" is still overloaded. A compute root, authority root, trust root, and ontological root can diverge; the framework must name which is under study.
- "Conditional" is promising but currently informal. Necessary, amplifying, and sufficient conditions must be separated and tested.
- The wishbone model is a useful visualization, not yet an established law. Branching, binary choice, rigidity, scale, feedback, and catastrophic transition are distinct hypotheses requiring separate experiments.
- Base-reality compromise lacks an operational definition. State corruption, rule change, operator replacement, resource loss, and ontological failure must not be merged.
- A universal map of reality is outside the stated goal and not supportable. The deliverable should be a bounded, revisable model with explicit unknowns.
- Redundancy can create common-mode failure and hidden complexity. Diversity, independence, and recovery testing matter more than replica count.
- A single "root cause" framing may oversimplify interacting conditions. Fault trees and causal-factor analysis should preserve combinations and enabling conditions [R11][R20].
- The Department of Endings can itself become a root dependency, a political bottleneck, or a failure theater unless its mandate, oversight, evidence standards, and feedback authority are defined.

Priority research questions

What exact functions can be root-level, and can several roots coexist for compute, time, identity, authority, history, and recovery?

Which dimensions of the rootness vector are necessary for CFTRL, and which merely increase severity?

What observations can discriminate an inherited fault from a local fault under each communication and timing model?

What is the minimum independent witness needed to detect a compromised evidence plane?

How should "legitimate state" be defined if the root can rewrite the rules that define legitimacy?

When does redundancy reduce risk, and when does it create shared assumptions or a new higher-order root?

Can objective-function drift be inferred from allocation behavior without privileged access?

Which failure classes are reversible, and which destroy the information required for recovery?

- How should uncertainty be reported when several root models are observationally equivalent?
- What evidence would falsify each proposed root model or failure signature?
- How does the framework treat emergent foundations that were not intentionally designed as roots?
- Can a system migrate foundational functions without losing identity or continuity?
- What is the safe survival envelope for dependents when parental maintenance stops?
- How are laboratory findings validated against real but non-cosmological analogues such as virtualization, infrastructure, organizations, and safety systems?
- What rights and duties does a root owe to dependent layers, especially when termination, scarcity, or evidence control are asymmetric?

Recommended direction

- Freeze and hash the source corpus; remove unrelated contamination from working copies while preserving the untouched original record.
- Approve a controlled glossary and boundary declaration template before writing code.
- Adopt the rootness vector and evidence ladder provisionally, then test whether they discriminate real cases.
- Approve a failure taxonomy v0.1 and map every class to at least one falsifiable experiment.
- Build the minimum Root Identification Blind Test with synthetic agents and independent ground truth.
- Validate the harness using known engineering cases before extending claims to reality-level structures.
- Charter the Department of Endings as a research and assurance function with limited authority, transparent records, and required design-response tracking.
- Separate three publication tracks: engineered systems findings, simulation-model findings, and philosophical/ontological analysis.
- Use verification and validation gates before any result enters approved doctrine [R10].
- Keep all outputs in draft status until Architect review and ONESTAMP.

Conclusion

CFTRL is ready to become a disciplined research program, but it is not yet a validated framework. Its highest-value contribution is a precise study of embedded diagnosis under asymmetric dependency, observability, authority, and repair access. The next decisive step is not a broader cosmological claim; it is a bounded laboratory that can measure when lower layers correctly identify—or confidently misidentify—the source of failure. Success will be demonstrated by reproducible distinctions, calibrated uncertainty, safer architectures, preserved evidence, and documented learning from endings.

Research evidence register

The following sources support the research layer. They are external evidence, not OAL canon. Inline markers [R#] identify the source supporting a claim. Source descriptions are intentionally limited to what the cited material establishes.

ID	Source	Relevance	URL
R1	NIST CSRC, "Trusted Computing Base (TCB)"	Defines the TCB as the totality of hardware, firmware, and software protection mechanisms responsible for enforcing a security policy.	https://csrc.nist.gov/glossary/term/trusted_computing_base
R2	NIST, Hardware-Enabled Security / Platform Integrity	Describes cryptographic measurement, chains of trust, roots of trust for update, detection, and recovery, and remote attestation.	https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=934427
R3	NIST SP 800-160 Vol. 2 Rev. 1, Developing Cyber-Resilient Systems	Frames resilience as anticipating, withstanding, recovering from, and adapting to adverse conditions, stresses, attacks, and compromises across the system life cycle.	https://csrc.nist.gov/pubs/sp/800/160/v2/r1/final
R4	Google SRE, "Addressing Cascading Failures"	Shows how overload, feedback, and secondary symptoms can spread failures and make apparent causes misleading.	https://sre.google/sre-book/addressing-cascading-failures/
R5	Buldyrev et al., Nature 464 (2010), "Catastrophic cascade of failures in interdependent networks"	Demonstrates that dependency coupling can propagate small failures recursively across networks and produce fragmentation.	https://www.nature.com/articles/nature08932

ID	Source	Relevance	URL
R6	E. W. Dijkstra, "Self-stabilizing Systems in Spite of Distributed Control" (1974)	Defines self-stabilization as eventual return to a legitimate state regardless of initial state, under the stated model.	https://courses.csail.mit.edu/6.852/05/papers/p643-Dijkstra.pdf
R7	Lamport, Shostak, and Pease, "The Byzantine Generals Problem" (1982)	Formalizes agreement when components may send conflicting information and establishes fault-tolerance limits under specified communication assumptions.	https://dl.acm.org/doi/pdf/10.1145/357172.357176
R8	Fischer, Lynch, and Paterson, "Impossibility of Distributed Consensus with One Faulty Process" (1985)	Shows that deterministic consensus cannot be guaranteed in a fully asynchronous system even with one crash fault under the paper's assumptions.	https://www.cs.princeton.edu/courses/archive/fall07/cos518/papers/flp.pdf
R9	Stanford Encyclopedia of Philosophy, "Causal Models"	Explains that causal models support predictions about interventions and counterfactuals, not only observed correlations.	https://plato.stanford.edu/entries/causal-models/
R10	NASA Systems Engineering Handbook / Expanded Guidance	Separates verification ("built right") from validation ("right product") and requires planning, execution, analysis, reporting, and work-product capture.	https://www.nasa.gov/reference/systems-engineering-handbook/
R11	NASA Software Engineering Handbook, "Software Fault Tree Analysis"	Describes deductive analysis from an undesired top event to credible combinations of contributing faults or system states.	https://swehb.nasa.gov/spaces/SWEHBVC/pages/72025030/8.07+-+Software+Fault+Tree+Analysis
R12	CISA, Shields Ready	Recommends identifying critical assets, mapping dependencies, assessing risks, exercising plans, and adapting from lessons learned.	https://www.cisa.gov/shields-ready
R13	AWS Prescriptive Guidance, Chaos Engineering	Distinguishes deterministic resilience testing from hypothesis-based chaos experiments; emphasizes baselines, stop conditions, limited blast radius, monitoring, and learning.	https://docs.aws.amazon.com/prescriptive-guidance/latest/chaos-engineering-on-aws/overview.html
R14	Nick Bostrom, "Are We Living in a Computer Simulation?" (2003)	Presents a probabilistic trilemma; it does not itself provide a direct internal test for identifying base reality.	https://academic.oup.com/pq/article-abstract/53/21/1243/1610975
R15	David Chalmers, discussion of perfect and imperfect simulations (2022)	Argues that an imperfect simulation might provide detectable evidence, while a perfect simulation may be observationally indistinguishable from non-simulated reality.	https://www.carnegiecouncil.org/media/series/aiei/how-real-is-virtual-reality-david-chalmers
R16	Vazza, "Astrophysical constraints on the simulation hypothesis" (Frontiers in Physics, 2025)	Evaluates physical resource constraints for particular simulation scenarios and emphasizes that claims outside shared physical assumptions remain difficult or impossible to falsify.	https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2025.1561873/full
R17	NTSB, Safety Management Systems	Treats safety as an organization-wide cycle of policy, risk management, assurance, promotion, data collection, mitigation, and evaluation.	https://www.nts.gov/Advocacy/mwl/Pages/mwl-3.aspx
R18	INCOSE definition and systems-boundary discussion (IET summary)	Defines systems engineering as an integrative life-cycle approach and distinguishes the system boundary from its broader context.	https://www.theiet.org/impact-society/policy-and-public-affairs/digital-futures-policy/reports-and-papers/quantum-technologies-a-systems-approach
R19	NIST SP 800-207, Zero Trust Architecture	Rejects implicit trust based only on location or ownership and requires explicit authentication and authorization for access.	https://www.nist.gov/publications/zero-trust-architecture
R20	NASA Spaceflight Mishap Investigation Handbook	Uses timelines, fault trees, event-and-causal-factor trees, and recommendations to identify chains of events and conditions rather than stopping at a proximate cause.	https://www.nasa.gov/wp-content/uploads/2023/03/spaceflight-mishap-handbook-tagged.pdf